**BADDI UNIVERSITY OF EMERGING SCIENCES AND TECHNOLOGY**
**School of Engineering & Emerging Technologies (SEET)**
Department of Computer Science & Engineering • Makhnumajra, Baddi, District Solan (H.P.)

MINOR PROJECT (CSE)
Academic Year 2025–2026

SOFTWARE REQUIREMENTS SPECIFICATION & TECHNICAL DESIGN DOCUMENT (SRS / TDD)

# CampusKart

## Buy. Sell. Exchange. Connect.

An engineered cross-platform peer-to-peer campus marketplace designed for collegiate institutions, pairing real-time Firestore sync with server-side FastAPI and Groq LPU inference.

### DOCUMENT CONTROL & SYSTEM METADATA

| | | | |
|---|---|---|---|
| Project Name: | **CampusKart** | Specification Ref: | **BUEST-SEET-CS-2026** |
| Target Client: | **Android Application & Web Portal** | Baseline Version: | **v1.0 (Engineering Final)** |
| Project Guide: | **Asst. Prof. Ms. Shailja Sharma** | Review Status: | **SRS / TDD Signoff** |

### STUDENT DEVELOPMENT TEAM & ENGINEERING ALLOCATIONS

| Team Member | Roll Number | Designated Technical Role | Core Subsystem Scope |
|---|---|---|---|
| **Anurag Kumar Jha** LEAD | **23UCS102** | **Full Stack Flutter Developer** | Client Architecture, FastAPI Microservice & AI Integration |
| **Chandan Singh Bhandari** | **23UCS101** | Backend & Real-Time Systems Developer | Cloud Firestore NoSQL, Auth & Real-Time Chat Engine |
| **Manisha Kumari Sah** | **23UCS060** | UI/UX Designer & QA Engineer | Flutter Design System, Validation & Test Suite Execution |

## 1. EXECUTIVE SUMMARY & BACKGROUND

CampusKart is a localized, peer-to-peer commerce and exchange application specifically tailored to the university campus environment. In collegiate ecosystems like Baddi University of Emerging Sciences and Technology (BUEST), students face continuous turnover of semester-specific textbooks, engineering drafters, laboratory kits, scientific calculators, and dormitory essentials.

At present, these peer transactions occur informally via transient messaging channels (such as WhatsApp batches and Telegram channels) or physical notice boards. These channels suffer from extreme message churn, zero indexing, lack of price transparency, and critical privacy vulnerabilities where students are forced to publish their personal mobile numbers in public forums.

CampusKart eliminates these points of friction by deploying a dedicated platform engineered with **Flutter** for unified Android and Web delivery. The system features synchronized real-time buyer–seller messaging backed by **Cloud Firestore**, alongside low-latency generative AI assistance powered by the **Groq API** through a secure, self-hosted **Python FastAPI** middleware.

## 2. PROBLEM STATEMENT & SCOPE BOUNDARIES

| Deficiency Area | Current Informal Channel Shortfall | CampusKart Engineering Solution |
|---|---|---|
| **Information Indexing** | Listings disappear under general group chatter within hours; search is impossible. | Structured catalog with category filters, keyword indexing, and active condition tags. |
| **Student Privacy** | Sellers must publish personal phone numbers to open groups, risking harassment. | In-app 1-to-1 messaging via Firestore listeners, keeping phone numbers private. |
| **Listing Quality** | Listings lack critical specifications (wear, edition), triggering tedious back-and-forth. | FastAPI + Groq LPU module synthesizes polished, accurate listings from simple bullet points. |
| **Negotiation Stalling** | Uncertainty in fair student pricing causes prolonged messaging stalls. | One-tap contextual AI reply suggestions based on the last 3 conversation turns. |

### System Scope & Boundary Constraints

| Capability Domain | Functional Definition | Scope Classification |
|---|---|---|
| **Campus Catalog & Search** | Real-time querying, category filtering, and image galleries across Android & Web. | IN SCOPE (CORE MVP) |
| **In-App Chat Engine** | Two-party persistent message synchronization without external phone number sharing. | IN SCOPE (CORE MVP) |
| **AI Writing & Negotiation** | FastAPI proxy invoking Groq Llama-3 models for descriptions and reply chips. | IN SCOPE (CORE AI) |
| **Payment Processing** | Payment gateways, UPI escrow holding, and automated bank settlements. | OUT OF SCOPE (IN-PERSON HANDOFF) |
| **Inter-City Logistics** | Courier shipping integrations, tracking numbers, and parcel handling. | OUT OF SCOPE (ON-CAMPUS ONLY) |

## 3. FUNCTIONAL REQUIREMENTS SPECIFICATION (IEEE 830)

Requirements are systematically enumerated with unique traceability codes, categorized under the MoSCoW prioritization matrix for minor project evaluation.

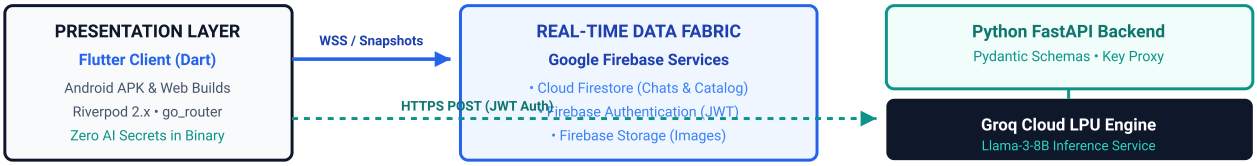
| Req ID | Functionality | Technical Specification & System Behavior | Priority |
|---|---|---|---|
| **FR-01** | Student Authentication | Firebase Auth email/password identity management. Profiles stored in Firestore `/users` collection with display name and roll number. | MUST HAVE |
| **FR-02** | Listing Creation | Sellers post items with title, price, category, condition (New, Good, Fair), description, and up to 4 compressed images in Firebase Storage. | MUST HAVE |
| **FR-03** | Search & Filters | Title text queries with multi-parameter filtering (Books, Electronics, Lab Gear, Furniture) and price sort filters. | MUST HAVE |
| **FR-04** | Real-Time In-App Chat | 1-to-1 conversation threads using Firestore sub-collection listeners. Messages contain senderId, content, timestamp, and read status. | MUST HAVE |
| **FR-05** | Item Watchlist | Buyers bookmark items to personal watchlists, updating state via Riverpod providers across active sessions. | SHOULD HAVE |
| **FR-06** | AI Listing Generator | Raw bullet points trigger an HTTPS POST to FastAPI, which queries Groq API (Llama-3-8B) to return a structured description. | MUST HAVE |
| **FR-07** | AI Reply Suggestions | Within active chat, recent message context is forwarded to FastAPI to generate 3 one-tap contextual responses (Accept, Counter, Decline). | SHOULD HAVE |
| **FR-08** | Listing Status Lifecycle | Sellers toggle status between `AVAILABLE`, `RESERVED`, and `SOLD`. Sold items are filtered out of public feeds. | MUST HAVE |

## 4. NON-FUNCTIONAL REQUIREMENTS SPECIFICATION (NFR)

| NFR ID | Quality Attribute | Target Metric & Engineering Standard | Verification Method |
|---|---|---|---|
| **NFR-01** | Chat Propagation Speed | Message delivery between active connected clients must complete within **< 500 ms** under steady campus Wi-Fi / 4G. | Firestore WebSocket listener telemetry |
| **NFR-02** | AI Inference Latency | End-to-end roundtrip for AI listing generation must return within **< 1500 ms**. | Groq LPU benchmark via FastAPI |
| **NFR-03** | Client Security | Zero third-party API secret keys (Groq Key) packaged inside client Android APK or Web builds. | Static analysis audit of compiled artifacts |
| **NFR-04** | Data Access Isolation | Firestore Security Rules must restrict document edits strictly to the document owner (seller UID validation). | Firebase Emulator test suite |
| **NFR-05** | Cross-Platform UI | Fluid layout scaling from Android mobile displays (360dp–480dp) to desktop browser viewports (≥ 1024px). | Flutter layout tests & physical devices |
| **NFR-06** | Storage Optimization | Client-side image compression reducing uploads to ≤ 300 KB per photo before uploading to Firebase Storage. | Image processing pipeline audit |

## 5. HIGH-LEVEL SYSTEM ARCHITECTURE & COMPONENT INTERACTION

CampusKart adopts a multi-tier decoupled architecture separating the Flutter client, real-time data storage, and the AI microservice pipeline to guarantee client security and maintainable code boundaries.




### Architectural Invariants & Separation Principles

**1. Real-Time Storage vs. AI Inference Decoupling**

Cloud Firestore is the sole database and message broker. Groq API is strictly an ephemeral natural language processor. **Groq does not store, forward, or log student chat histories.** Real-time messaging runs independently of AI availability.

**2. Strict Server-Side Secret Key Isolation**

Under zero circumstances is the Groq API key compiled into the Flutter mobile APK or Web bundle. All generative calls route through the self-hosted FastAPI backend, which injects the key on the server side after validating client tokens.

**System Communication Protocols**

- **Client-to-Firestore:** Persistent Secure WebSockets (WSS) and gRPC streaming for sub-second snapshot synchronization.
- **Client-to-FastAPI:** Asynchronous HTTPS REST calls with Pydantic payload validation and JSON serialization.
- **FastAPI-to-Groq:** Encrypted HTTPS calls utilizing the official Python `AsyncGroq` client with explicit timeout controls.

## 6. PRODUCTION FASTAPI BACKEND MICROSERVICE SPECIFICATION

The AI backend is implemented as a lightweight, non-blocking asynchronous Python FastAPI service. It handles input sanitization, rate management, and secure API key injection before querying the Groq LPU inference cloud.

### 6.1 Pydantic Request/Response Models (`app/schemas/ai_schemas.py`)

```
from pydantic import BaseModel, Field from typing import List, Optional class
GenerateDescriptionRequest(BaseModel): item_title: str = Field(..., min_length=3, max_length=80,
example="Engg Mathematics - HK Dass") category: str = Field(..., example="Books & Academics") condition:
str = Field(..., example="Like New, 3rd Sem") price: float = Field(..., gt=0, example=250.0)
additional_notes: Optional[str] = Field(None, max_length=200, example="Hardcover, no marks inside")
class GenerateDescriptionResponse(BaseModel): enhanced_title: str generated_description: str
suggested_tags: List[str] class SmartReplyRequest(BaseModel): last_message: str = Field(...,
min_length=1, max_length=300, example="Can you do 200 INR?") item_title: str = Field(...,
example="Scientific Calculator FX-991EX") item_price: float = Field(..., example=600.0) is_seller: bool
= True class SmartReplyResponse(BaseModel): suggestions: List[str]
```

### 6.2 Asynchronous Endpoint Handlers (`app/main.py`)

```
import os, json from fastapi import FastAPI, HTTPException, status from fastapi.middleware.cors
import CORSMiddleware from groq import AsyncGroq from app.schemas.ai_schemas import (
GenerateDescriptionRequest, GenerateDescriptionResponse, SmartReplyRequest, SmartReplyResponse ) app =
FastAPI(title="CampusKart AI Backend", version="1.0.0") app.add_middleware(CORSMiddleware,
allow_origins=["*"], allow_methods=["*"], allow_headers=["*"]) groq_client =
AsyncGroq(api_key=os.getenv("GROQ_API_KEY")) @app.post("/api/v1/ai/generate-description",
response_model=GenerateDescriptionResponse) async def generate_description(payload:
GenerateDescriptionRequest): system_prompt = ( "You are an assistant for CampusKart at BUEST. Generate a
crisp, honest, student-friendly " "description. Return strictly a JSON object with keys:
'enhanced_title', 'generated_description', 'suggested_tags'." ) user_prompt = f"Item:
{payload.item_title}, Category: {payload.category}, Condition: {payload.condition}, Price:
{payload.price}" try: res = await groq_client.chat.completions.create( model="llama-3.1-8b-instant",
messages=[{"role": "system", "content": system_prompt}, {"role": "user", "content": user_prompt}],
temperature=0.3, max_tokens=350, response_format={"type": "json_object"} ) data =
json.loads(res.choices[0].message.content) return GenerateDescriptionResponse(**data) except Exception
as e: raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail=str(e))
```

## 7. DATABASE DESIGN & CLOUD FIRESTORE DATA MODEL

Cloud Firestore organizes application data into top-level document collections and structured sub-collections. The schema is denormalized to optimize read performance and minimize document query counts during marketplace browsing.

**Collection: `/users/{uid}`**

| | | |
|---|---|---|
| **uid** | String (PK) | Firebase Auth UID |
| **email** | String | Campus email address |
| **displayName** | String | Student full name |
| **rollNumber** | String | University roll number |
| **createdAt** | Timestamp | Registration timestamp |
| **favorites** | Array<String> | Bookmarked listing IDs |

**Collection: `/products/{productId}`**

| | | |
|---|---|---|
| **productId** | String (PK) | Auto-generated UUID |
| **sellerId** | String (FK) | Reference to `users.uid` |
| **title** | String | Listing headline |
| **price** | Number | Asking price in INR (₹) |
| **category** | String | Books / Kits / Gadgets / Other |
| **condition** | String | "New" \| "Good" \| "Fair" |
| **status** | String | "AVAILABLE" \| "RESERVED" \| "SOLD" |
| **imageUrls** | Array<String> | Storage media download URLs |

**Real-Time Chat Hierarchy: `/chats/{chatId}` and Sub-Collection `/messages/{msgId}`**

**Thread Document: `/chats/{chatId}`**

| | | |
|---|---|---|
| **participants** | Array<String> | [buyerUid, sellerUid] |
| **productId** | String | Target product reference |
| **lastMessage** | String | Snippet for inbox preview |
| **lastUpdated** | Timestamp | Index key for sorting feed |

**Sub-Collection: `.../messages/{msgId}`**

| | | |
|---|---|---|
| **senderId** | String | UID of the author |
| **content** | String | Message text string |
| **sentAt** | Timestamp | Chronological ordering key |
| **isRead** | Boolean | Status indicator for receipts |

## 8. FIRESTORE DECLARATIVE SECURITY RULES MATRIX

```
rules_version = '2'; service cloud.firestore { match /databases/{database}/documents { match
/users/{userId} { allow read: if request.auth != null; allow write: if request.auth != null &&
request.auth.uid == userId; } match /products/{productId} { allow read: if request.auth != null; allow
create: if request.auth != null && request.resource.data.sellerId == request.auth.uid; allow update,
delete: if request.auth != null && resource.data.sellerId == request.auth.uid; } match /chats/{chatId} {
allow read, write: if request.auth != null && request.auth.uid in resource.data.participants; allow
create: if request.auth != null && request.auth.uid in request.resource.data.participants; match
/messages/{messageId} { allow read, write: if request.auth != null && request.auth.uid in
get(/databases/$(database)/documents/chats/$(chatId)).data.participants; } } } }
```

## 9. UI/UX DESIGN SYSTEM & NAVIGATION STRUCTURE

CampusKart adopts an uncluttered, high-contrast visual design system. The interface avoids unnecessary decorative assets in favor of scannable data layouts and standard typography.

**Primary Slate (#0F172A)**
Top app bars, primary actions, heavy typography, and structural borders.

**Brand Blue (#2563EB)**
Active tab states, interactive buttons, links, and outgoing chat bubbles.

**Accent Teal (#0D9488)**
AI feature indicators, verification badges, and success status banners.

### Screen Inventory & Declarative Navigation (`go_router`)

| Route Path | Screen Component | User Actions & Interface Widgets | State Scope |
|---|---|---|---|
| `/auth` | Authentication Screen | Sign-in / sign-up forms, campus email validation, password reset action. | `authProvider` |
| `/home` | Marketplace Feed | Search bar, category pill filters, 2-column masonry grid, pull-to-refresh. | `feedProvider` |
| `/product/:id` | Item Detail Screen | Image carousel, price badge, seller meta info, 'Message Seller' floating CTA. | `productFamily` |
| `/create-listing` | Create Listing Screen | Multi-photo picker, price field, condition dropdown, 'Enhance with AI' button. | `listingForm` |
| `/chats` | Conversations Inbox | Active buyer/seller chat threads, unread counters, last message snippet. | `inboxStream` |
| `/chat/:chatId` | Direct Messaging Screen | Firestore real-time message stream, text input bar, Groq AI quick reply chips. | `chatStream` |

## 10. USER JOURNEY & TRANSACTION LIFE CYCLE

**Buyer Transaction Journey**

1. **Discovery:** Filter items by category (e.g., Books) and search title keywords.
2. **Inspection:** Review photos, condition tag, asking price, and seller location.
3. **Direct Inquiry:** Tap 'Message Seller' to open a private Firestore chat thread.
4. **Negotiation:** Coordinate price and propose physical campus meetup location.
5. **Exchange:** Meet in person on campus (library, cafeteria) and complete cash/UPI trade.

**Seller Transaction Journey**

1. **Drafting:** Enter basic item title, condition, asking price, and upload 1–4 photos.
2. **AI Polish:** Tap 'Enhance with AI' to generate a structured description via FastAPI.
3. **Publication:** Submit listing; item immediately appears in public feed.
4. **Communication:** Receive buyer messages; use one-tap AI reply chips to negotiate.
5. **Resolution:** Mark item as `SOLD` once physical transaction concludes.

## 11. FLUTTER CLIENT IMPLEMENTATION & STATE ARCHITECTURE

The frontend leverages **Flutter 3.48** and **Dart 3.5**, targeting Android mobile devices and modern desktop web browsers from a single codebase. State management is orchestrated using **Flutter Riverpod 2.5**.

**Riverpod Reactive State Management Strategy**

- `StreamProvider.autoDispose`: Subscribes directly to Firestore snapshots for real-time chat threads. Closes WebSocket channels automatically when users navigate away, eliminating memory leaks.
- `AsyncNotifierProvider`: Manages asynchronous side-effects such as calling the FastAPI AI proxy, handling loading states, and showing error snackbars.
- `StateProvider`: Manages local UI filters (selected category, price sort order, search query string).

## 12. TECHNOLOGY STACK SELECTION & TECHNICAL JUSTIFICATION

| Technology | Architectural Role | Engineering Justification |
|---|---|---|
| **Flutter & Dart** | Cross-Platform Client | Single codebase compiles to native ARM Android machine code and WebAssembly/HTML5 web builds, eliminating cross-platform maintenance divergence. |
| **Flutter Riverpod** | State Management | Compile-time safe, decoupled dependency injection pattern. Eliminates mutable global state anti-patterns and context coupling. |
| **go_router** | Declarative Router | Enables deep-linking for both mobile notifications and clean browser URL paths, providing declarative authentication redirect guards. |
| **Cloud Firestore** | NoSQL Database | Managed document store with built-in WebSocket snapshot listeners, delivering low-latency real-time chat updates out of the box. |
| **Python FastAPI** | AI Microservice Backend | High-performance ASGI framework with native Pydantic validation, facilitating non-blocking asynchronous calls to the Groq LPU engine. |
| **Groq API (LPU)** | AI Inference Engine | Language Processing Units (LPUs) provide low-latency token generation for Llama-3 models, ensuring AI replies generate in under 1.5 seconds. |

**Client-Side Image Optimization Pipeline**

Before upload to Firebase Storage, photos are resized client-side to a max width of 1280px and compressed via `flutter_image_compress` to ≤ 300 KB. This limits storage footprint, reduces student mobile data consumption, and improves feed load times.

## 13. VERIFICATION, TESTING & QUALITY ASSURANCE

A structured testing suite ensures the reliability, responsiveness, and security of both the client interface and backend services across all deployment targets.

| Test Tier | Scope & Tooling | Verification Scenario | Pass Metric |
|---|---|---|---|
| **Unit Testing** | Flutter Test & Pytest | Pydantic schema validation, regex phone number stripping, Riverpod state mutations. | 100% pass on core helpers |
| **Integration Testing** | Firebase Local Emulator | Dual-client chat synchronization, message ordering, unread badge counter logic. | Latency < 500ms; zero dropped messages |
| **AI Proxy Resilience** | FastAPI Pytest / Mocking | Behavior under simulated Groq timeouts, malformed inputs, and API rate limits. | Graceful fallback error; HTTP 200 within 1.5s |
| **UI Responsiveness** | Chrome DevTools & Android Devices | UI layout verification across mobile screen widths (360dp–480dp) and desktop displays. | Zero visual overflow errors |

## 14. RISK ASSESSMENT & MITIGATION ENGINEERING

| Identified Risk | Severity | Potential Impact | Technical Mitigation |
|---|---|---|---|
| **Groq API Rate Limits** | HIGH | AI description generation or smart replies fail during demonstrations. | Client retry with exponential backoff; manual fallback editing option. |
| **Firestore Read Quotas** | MEDIUM | Uncached marketplace browsing exhausts Firebase free-tier read limits. | Query pagination (20 items/page) and Riverpod local memory cache. |
| **Inappropriate Listings** | MEDIUM | Students posting prohibited or non-academic items on the feed. | Keyword sanitization in FastAPI and in-app listing report flags. |
| **Unstable Wi-Fi** | LOW | Intermittent campus connectivity disrupts real-time message sending. | Firestore offline persistence queues outbound messages locally. |

### Target Deployment & Distribution Plan

• **Android Release:** Release-signed APK package (`flutter build apk --release`) distributed directly to student testers across campus hostels.
• **Web Portal:** Hosted via **Firebase Hosting** (`flutter build web`) offering zero-install access on desktop and laptop browsers.
• **FastAPI Backend:** Containerized with Docker and deployed to a cloud runtime (Render / Railway) with environment variable isolation.

## 15. PROJECT ROADMAP & AGILE MILESTONE SCHEDULE

Development follows a bi-weekly iterative schedule aligned with SEET departmental review cycles.

| Milestone | Work Package | Deliverables & Verification Standard | Status |
|---|---|---|---|
| **Phase 1**<br>Weeks 1–2 | Architecture & Scaffolding | SRS/TDD document baseline, Riverpod structure, Firebase setup, and Auth workflows. | COMPLETED |
| **Phase 2**<br>Weeks 3–4 | Feed & Chat Sync | Listing CRUD, Firestore chat streams, image upload pipeline, and category filtering. | IN PROGRESS |
| **Phase 3**<br>Weeks 5–6 | FastAPI & AI Integration | FastAPI deployment, Groq LPU pipeline, AI listing enhancer, and smart reply chips. | SCHEDULED |
| **Phase 4**<br>Weeks 7–8 | Integration & Defense | Cross-device QA testing, APK distribution, Web portal deployment, and project defense. | SCHEDULED |

## 16. TECHNICAL TEAM RESPONSIBILITY MATRIX (RACI)

| Team Member & Roll | Designated Technical Role | Core Responsibilities & Assigned Deliverables |
|---|---|---|
| **Anurag Kumar Jha**<br>Roll No. 23UCS102 | **Full Stack Flutter Developer**<br>(Team Lead) | Flutter cross-platform client architecture, Riverpod state models, FastAPI backend microservice, Groq prompt engineering, and end-to-end integration. Cloud Firestore data layer, real-time chat listeners, Firebase Authentication rules, media storage pipeline, and local emulator integration testing. |
| **Chandan Singh Bhandari**<br>Roll No. 23UCS101 | AI Module Developer | Responsible for integrating AI features into CampusKart AI, including AI-powered listing descriptions, smart search assistance, and AI-generated reply suggestions using the Groq API and Python FastAPI backend. |
| **Manisha Kumari Sah**<br>Roll No. 23UCS060 | UI/UX Designer &<br>QA Engineer | Flutter UI component library, input validation rules, cross-platform layout responsiveness, test case authoring, and defect tracking. |

## 17. ACADEMIC EVALUATION & DEPARTMENTAL ASSESSMENT GATES

| Review Stage | Evaluation Scope & Deliverables | Guide Evaluation Status |
|---|---|---|
| **Review 1 (Proposal & SRS)** | Problem formulation, feasibility, architecture, SRS & TDD baseline signoff. | **SRS/TDD Baseline Ready** |
| **Review 2 (Mid-Term MVP)** | Working MVP: Firebase Auth, Listing CRUD, and Firestore real-time messaging. | *Scheduled Week 4* |
| **Review 3 (Final Defense)** | FastAPI Groq AI integration, Android APK testing, Web deployment, final code review. | *Scheduled Week 8* |

Project Guide: **Assistant Professor Ms. Shailja Sharma** • Department of Computer Science & Engineering, BUEST